PROJET PYTHON – MODULE TURTLE

Exercice 1 : La fleur à pétales

Analyse du code et réponses aux questions de réflexion

Nom	Muteba Wa Muteba
Prénom	Israël
Classe	L2 (CS)
Fichier analysé	`projet(exo1).py`
Date	12 novembre 2026

1. Présentation de l’exercice

Ce programme dessine une **fleur colorée** à l’aide du module `turtle` de Python. L’utilisateur choisit le nombre de pétales, leur taille et trois couleurs (en anglais). Les pétales sont ensuite tracés l’un après l’autre autour d’un même centre, sur un fond noir, en alternant les trois couleurs.

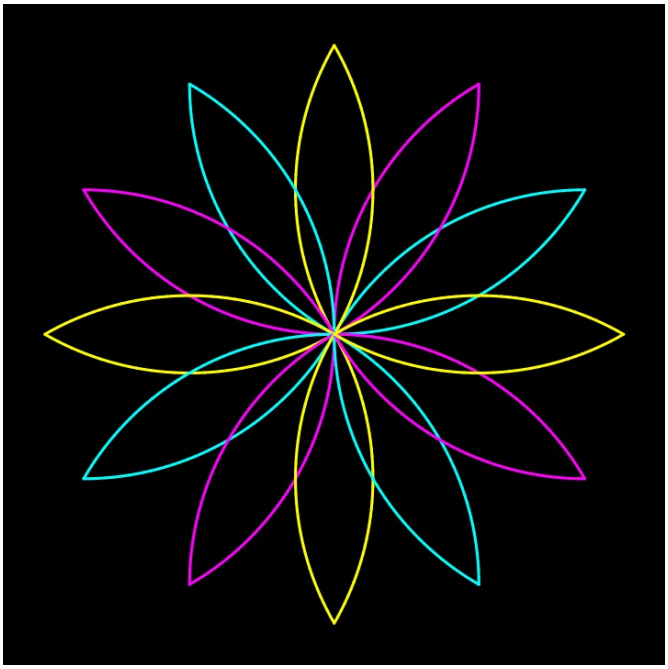


Figure 1 – *Résultat du programme avec nombre = 12, taille = 100 et les couleurs cyan, magenta et yellow (illustration)*

2. Code source analysé

Fichier : projet(exo1).py

```
import turtle

nombre = int(input("Combien de pétales voulez-vous ? : "))
taille = int(input("Quelle taille pour les pétales ? : "))

couleur1 = input("Première couleur(En anglais) : ")
couleur2 = input("Deuxième couleur(En anglais) : ")
couleur3 = input("Troisième couleur(En anglais) : ")

t = turtle.Turtle()
t.speed(0)

turtle.bgcolor("black")

def petale():
    t.circle(taille, 60)
    t.left(120)
    t.circle(taille, 60)
    t.left(120)

for i in range(nombre):
    if i % 3 == 0:
        t.color(couleur1)
    elif i % 3 == 1:
        t.color(couleur2)
    else:
        t.color(couleur3)

    petale()
    t.left(360 / nombre)

turtle.done()
```

Fonctionnement pas à pas

1. **Saisie des données** : `input()` lit le texte tapé au clavier et `int()` le convertit en nombre entier pour `nombre` et `taille`. Les couleurs restent du texte.
2. **Préparation** : `turtle.Turtle()` crée la tortue `t`, `t.speed(0)` règle la vitesse maximale et `turtle.bgcolor("black")` colore le fond en noir.
3. **Fonction** `petale()` : elle trace deux arcs de cercle de 60° (`t.circle(taille, 60)`) séparés par une rotation de 120°. Les deux arcs forment un pétale en forme de feuille et la tortue retrouve sa direction de départ (60 + 120 + 60 + 120 = 360°).
4. **Boucle principale** : pour chaque pétale, la couleur est choisie selon le reste de `i` divisé par 3, le pétale est dessiné, puis la tortue pivote de `360 / nombre` degrés pour préparer le suivant.
5. **Fin** : `turtle.done()` garde la fenêtre ouverte une fois le dessin terminé.

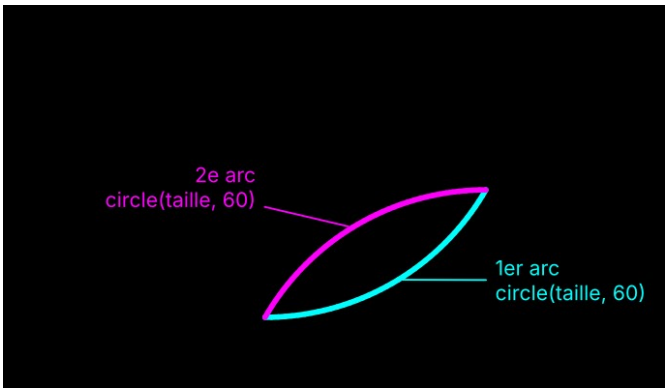


Figure 2 – *Un pétale seul : deux arcs de 60° (cyan puis magenta), chacun suivi d'une rotation de 120°*

3. Réponses aux questions d'analyse

Question 1 – Quelles instructions sont répétées ? Pourrait-on utiliser une boucle ?

Plusieurs répétitions apparaissent dans le programme.

a) Dans la fonction `petale()`

Le couple `t.circle(taille, 60)` puis `t.left(120)` est écrit deux fois de suite. Une boucle exécutée 2 fois suffit.

Avant

```
def petale():
    t.circle(taille, 60)
    t.left(120)
    t.circle(taille, 60)
    t.left(120)
```

Après

```
def petale():
    for _ in range(2):
        t.circle(taille, 60)
        t.left(120)
```

b) Dans les trois saisies de couleurs

Les trois lignes `input()` ne diffèrent que par leur numéro. Une boucle qui remplit une liste les remplace.

Proposition

```
couleurs = []
for n in range(3):
    couleurs.append(input("Couleur " + str(n + 1) + " (en anglais) : "))
```

c) Dans le choix de la couleur

Le bloc `if / elif / else` répète trois fois l'instruction `t.color(...)`. Avec la liste précédente, une seule ligne suffit :

Proposition

```
t.color(couleurs[i % len(couleurs)])
```

Cette ligne remplace six lignes et reste valable avec 3, 4 ou 10 couleurs.

d) La boucle principale

La répétition la plus importante est déjà traitée par `for i in range(nombre)`. Sans elle, il faudrait écrire `petale()` puis `t.left(360 / nombre)` autant de fois que de pétales, ce qui est impossible puisque ce nombre est choisi par l'utilisateur au moment de l'exécution.

Version simplifiée complète

Programme simplifié (même dessin)

```
import turtle

nombre = int(input("Combien de pétales voulez-vous ? : "))
taille = int(input("Quelle taille pour les pétales ? : "))

couleurs = []
for n in range(3):
    couleurs.append(input("Couleur " + str(n + 1) + " (en anglais) : "))

t = turtle.Turtle()
t.speed(0)
turtle.bgcolor("black")

def petale():
    for _ in range(2):
        t.circle(taille, 60)
        t.left(120)

for i in range(nombre):
    t.color(couleurs[i % len(couleurs)])
    petale()
    t.left(360 / nombre)

turtle.done()
```

> **À retenir** : dès qu'un groupe d'instructions se répète avec de petites variations, on cherche une boucle (`for`) ou une liste que l'on parcourt avec un indice.

Question 2 – Une partie du dessin pourrait-elle être isolée dans une fonction ?

Oui, et c'est déjà en partie le cas : la fonction `petale()` isole le motif qui se répète. Le corps du programme n'a plus qu'à l'appeler dans la boucle, ce qui rend le code plus lisible.

Trois améliorations sont possibles.

- **Passer la taille en paramètre** : `petale()` utilise actuellement la variable globale `taille`. Avec `def petale(taille):`, la fonction devient indépendante du reste du programme et réutilisable.
- **Isoler le choix de la couleur** dans une petite fonction, par exemple `choisir_couleur(i)`.
- **Isoler la fleur entière** dans une fonction `fleur(nombre, taille, couleurs)` pour pouvoir dessiner plusieurs fleurs.

Proposition : une fonction pour la fleur entière

```
def petale(taille):
    for _ in range(2):
        t.circle(taille, 60)
        t.left(120)

def fleur(nombre, taille, couleurs):
    for i in range(nombre):
        t.color(couleurs[i % len(couleurs)])
        petale(taille)
        t.left(360 / nombre)

fleur(12, 100, ["cyan", "magenta", "yellow"])
fleur(6, 50, ["red", "orange", "yellow"])   # 2e fleur, plus petite
```

Les avantages sont la **lisibilité** (chaque fonction a un rôle précis), la **réutilisation** (on appelle la même fonction avec d'autres valeurs) et la **facilité de correction** (une erreur se cherche dans une seule fonction).

Question 3 – Quels paramètres pouvez-vous modifier pour changer le résultat ?

Paramètre	Où le modifier	Effet sur le dessin
`nombre`	Saisie au clavier	Nombre de pétales. L'écart entre deux pétales est de `360 / nombre` degrés.
`taille`	Saisie au clavier	Rayon des arcs. Avec des arcs de 60°, la longueur d'un pétale (de sa base à sa pointe) est égale à `taille`.
`couleur1`, `couleur2`, `couleur3`	Saisie au clavier	Couleurs des pétales, utilisées à tour de rôle.
`60` et `120`	Dans `petale()`	Forme du pétale : un arc de 60° donne un pétale étroit, un arc de 90° un pétale plus large.
`t.speed()`, `bgcolor()`, `pensize()`	Début du programme	Vitesse de tracé, couleur du fond, épaisseur du trait.

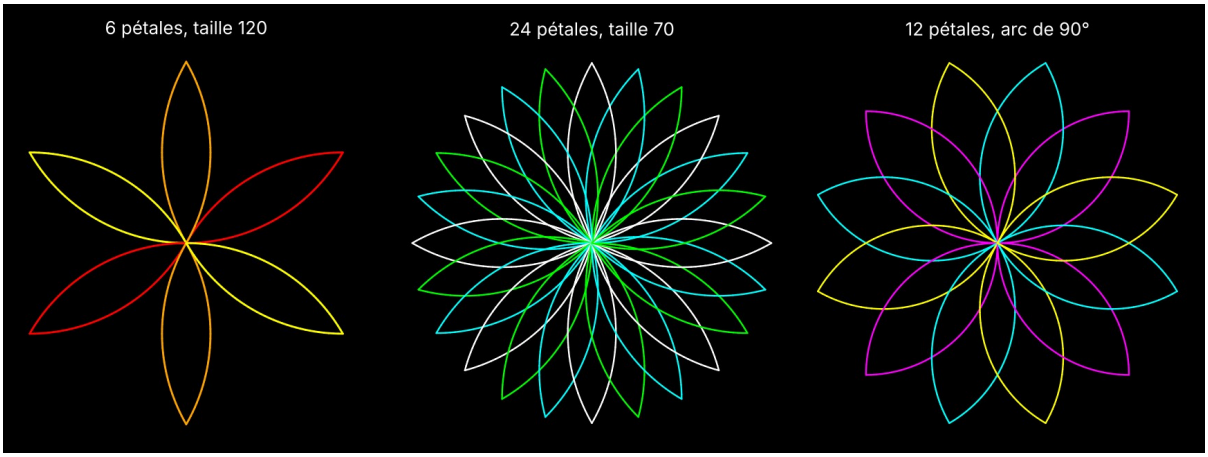


Figure 3 – *Effet des paramètres : 6 pétales de taille 120 (gauche), 24 pétales de taille 70 (centre), arcs de 90° au lieu de 60° (droite)*

Pour que le pétale se referme et que la tortue retrouve sa direction, il faut respecter la règle **2 × (arc + rotation) = 360°**. Avec un arc de 90°, la rotation doit donc être de 90° :

Pétale plus large

```
def petale():
    for _ in range(2):
        t.circle(taille, 90)
        t.left(90)
```

Question 4 – Quelles nouvelles instructions ou notions avez-vous dû découvrir ?

Notion	Explication	Exemple dans le code
Module `turtle`	Charger la bibliothèque de dessin et créer la « tortue » qui trace	`t = turtle.Turtle()`
Méthodes de la tortue	`speed()`, `color()`, `left()`, `bgcolor()`, `done()` pour piloter le dessin	`t.left(120)`
`circle(rayon, angle)`	Trace un arc de cercle : le 2e paramètre limite le tracé à une portion du cercle	`t.circle(taille, 60)`
`input()` et `int()`	`input()` renvoie toujours du texte ; `int()` le convertit en nombre entier	`int(input("..."))`
Fonction avec `def`	Regrouper des instructions sous un nom pour les réutiliser	`def petale():`
Opérateur modulo `%`	Reste d'une division, utilisé pour alterner 0, 1, 2, 0, 1, 2...	`i % 3`
`if / elif / else`	Choisir une action selon une condition	Choix de la couleur
`range(n)`	Répéter n fois avec un compteur `i` allant de 0 à n - 1	`for i in range(nombre)`
Noms de couleurs en anglais	Seuls les noms de couleurs anglais sont reconnus	`"cyan"`, `"yellow"`

Le modulo est la notion la plus importante pour l'alternance des couleurs :

Pétale `i`	0	1	2	3	4	5
`i % 3`	0	1	2	0	1	2
Couleur utilisée	couleur1	couleur2	couleur3	couleur1	couleur2	couleur3

Question 5 – Qu'avez-vous essayé lorsque le résultat n'était pas celui attendu ?

Problème observé	Hypothèse ou essai	Solution retenue
Les pétales ne se referment pas et la fleur part en spirale	Vérifier la somme des angles dans `petale()`. Essai avec `left(100)` à la place de `left(120)`	Retour à 60 + 120 + 60 + 120 = 360°
Message d'erreur sur la couleur (`bad color string`)	Les noms français (« rouge », « bleu ») ne sont pas connus de turtle	Utiliser les noms anglais : `red`, `blue`...

Problème observé	Hypothèse ou essai	Solution retenue
`ValueError` quand on tape du texte à la place d'un nombre	`int()` ne peut convertir que des chiffres	Saisir un entier, ou protéger la saisie avec `try / except`
Fleur trop grande ou trop petite	Tester plusieurs valeurs de `taille`	Choisir une taille adaptée à la fenêtre (environ 100)
Dessin trop lent	Le tracé animé prend du temps	`t.speed(0)` (vitesse maximale)

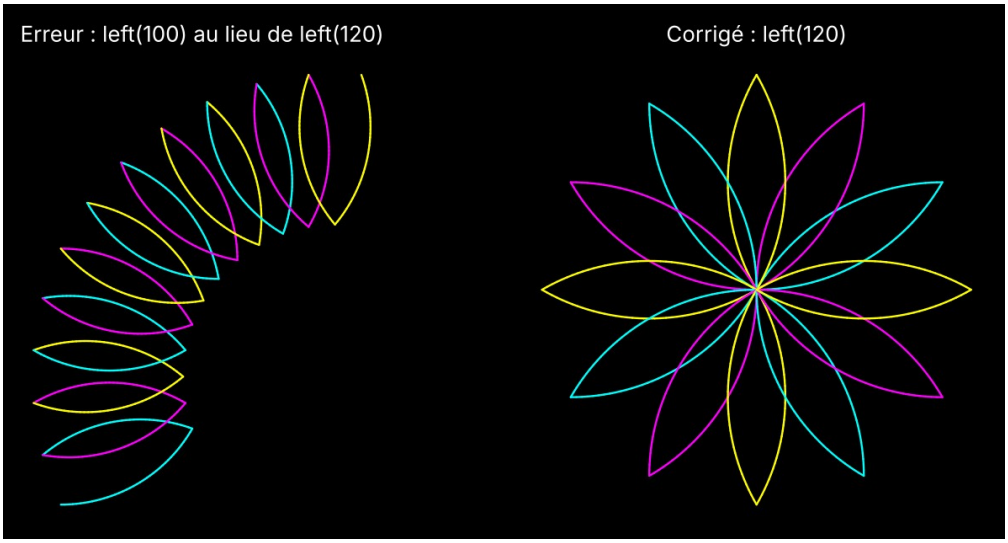


Figure 4 – *À gauche, l'erreur avec left(100) : les pétales ne se referment pas. À droite, le résultat corrigé avec left(120)*

Pour sécuriser la saisie du nombre de pétales, on peut utiliser :

Amélioration envisagée

```
try:
    nombre = int(input("Combien de pétales voulez-vous ? : "))
except ValueError:
    print("Veuillez taper un nombre entier.")
```

Méthode suivie : modifier **une seule valeur à la fois**, relancer le programme, observer l'effet, puis lire attentivement le message d'erreur (le numéro de ligne indiqué par Python est le meilleur point de départ).

4. Conclusion

Cet exercice montre comment un dessin complexe naît d'un **motif simple** (le pétale) **répété** et **tourné** régulièrement. Il m'a permis de comprendre l'intérêt des fonctions pour isoler un motif, des boucles pour le répéter, et du modulo pour alterner les couleurs. Les trois paramètres saisis par l'utilisateur (nombre, taille, couleurs) suffisent à produire des fleurs très différentes.

Document rédigé par **Muteba Wa Muteba Israël** – Classe L2 (CS).